

WEEK 1

Task1

Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that

1. Accept a Linux command as input.
2. Create a child process using `fork()`.
3. Execute the command in the child process using an appropriate `exec()` system call.
4. Allow the parent process to wait for the child using `wait()`.
5. Display the Process ID (PID) of both parent and child processes.

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

int main(){
    char command[100];
    pid_t pid;
    printf("Enter a Linux command: ");
    scanf("%s", command);
    pid = fork();
    if(pid < 0) {
        printf("Fork failed!\n");
        return 1;
    }
    else if(pid == 0) {
        printf("\nChild Process\n");
        printf("Child PID : %d\n", getpid());

        execlp(command, command, NULL);
        printf("Command execution failed.\n");
        exit(1);
    } else {
        wait(NULL);
        printf("\nParent Process\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child process completed.\n");
    }
    return 0;
}
```

Output:

```
Enter a Linux command: ls

Child Process
Child PID : 437
OSSP code.c files.c gm.c hello.c os program.c programming.c source_code
code files gm hello operating os.c programming programming.c.save source_code.c

Parent Process
Parent PID : 436
Child process completed.
```

Task2

Using Linux terminal commands (uname, lscpu, lsblk, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

Report

1. Uname: Displays operating system and kernel information.

```
balaji@BalajiPatil:~# uname
Linux
```

2. lscpu: Displays CPU information.

```
balaji@BalajiPatil:~# lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Address sizes:          39 bits physical, 48 bits virtual
Byte Order:             Little Endian
CPU(s):                 4
On-line CPU(s) list:    0-3
Vendor ID:              GenuineIntel
Model name:             Intel(R) Core(TM) i3-8145U CPU @ 2.10GHz
CPU family:             6
Model:                  142
Thread(s) per core:     2
Core(s) per socket:     2
Socket(s):              1
Stepping:               12
BogoMIPS:               4608.01
```

3. lsblk: Displays storage devices.

```
balaji@BalajiPatil:~# lsblk
NAME
    MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda      8:0    0 356.9M  1 disk
sdb      8:16    0 159.4M  1 disk
sdc      8:32    0    2G    0 disk [SWAP]
sdd      8:48    0    1T    0 disk /mnt/wslg/distro
/
```

4.ps: Displays currently running processes.

```
balaji@BalajiPatil:~# ps
      PID  TTY          TIME CMD
      352  pts/1        00:00:16 bash
      946  pts/0        00:00:00 ps
```

4.top: Displays real-time system performance.

```
balaji@BalajiPatil:~# top
top - 13:56:44 up 22 min,  1 user,  load average: 0.06, 0.03, 0.00
Tasks: 22 total,  1 running, 21 sleeping,  0 stopped,  0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem : 7880.6 total, 7463.6 free,  438.7 used,  123.2 buff/cache
MiB Swap: 2048.0 total, 2048.0 free,   0.0 used. 7441.9 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1	root	20	0	21680	13140	9744	S	0.0	0.2	0:00.74	systemd
2	root	20	0	3180	2200	2072	S	0.0	0.0	0:00.01	init-systemd(Ub
9	root	20	0	3196	2128	2008	S	0.0	0.0	0:00.00	init
64	root	19	-1	34052	12556	11488	S	0.0	0.2	0:00.25	systemd-journal
112	root	20	0	25548	6924	5168	S	0.0	0.1	0:00.26	systemd-udevd
123	systemd+	20	0	21344	13252	11124	S	0.0	0.2	0:00.12	systemd-resolve
124	systemd+	20	0	91036	7992	7024	S	0.0	0.1	0:00.10	systemd-timesyn
168	root	20	0	4244	2688	2432	S	0.0	0.0	0:00.00	cron
169	message+	20	0	9540	5432	4736	S	0.0	0.1	0:00.09	dbus-daemon
176	root	20	0	17980	8888	7848	S	0.0	0.1	0:00.08	systemd-logind
178	root	20	0	1756104	13024	10888	S	0.0	0.2	0:00.14	wsl-pro-service